

PermiCheck v2 Developer Manual

(short app description and purpose)

(tech stack info – Kotlin, AndroidX)

(client details)

1. Development Environment Setup

1.1 Required Tools:

- Android Studio Ladybug | 2024.2 or Later
- JDK 17 or Higher
- Kotlin 1.9+
- Gradle 8.0+
- Android SDK 34

1.2 Unpack & Setup

- Clone the project from the repository or extract the project from a .zip file
- Open the unpacked folder inside Android Studio:
 - File
 - Open
 - Select Folder -> Open
- Ensure grade sync completes successfully
- Set up local configuration using local.properties (SDK location)
- To run the application, select the emulator or a connected android device, then select run

1.3 Architecture Overview

Model-View-ViewModel architecture is used for PermiCheck with Jetpack Compose for user interface.

- **Model:** Data operations (DataStore, local storage).
- **View:** Composable UI elements which observe ViewModel state continuously and reactively
- **ViewModel:** Logic for user interactions, state flows.

2. Project Structure

app/



```
| | |— usage/      → Usage view model, daily app usage data, selectable date range
| | |— viewmodel/  → ViewModels controlling state and logic
| |
| |— App.kt        → Captures daily app usage periodically
|
|— build.gradle.kts → Gradle configuration for the app module
```

Room Database Implementation

Android's Room Persistence Library was utilised to manage local data storage. By using Room, we are able to store data related to permission changes and app usage even after the app restarts or runs offline.

This is crucial to how the 'Usage Tracking' and 'Recent Changes' sections of this app function. As there is no native API to access historical usage or permission data directly, the app now stores periodic snapshots of this information in Room database tables. These records allow the app to reconstruct and display this usage data and permission changes to the user.

Usage Pane Section

Purpose

The Usage Pane displays the user's app usage activity over time and displays the permissions these apps have. Information displayed includes total usage duration, session count and permissions granted.

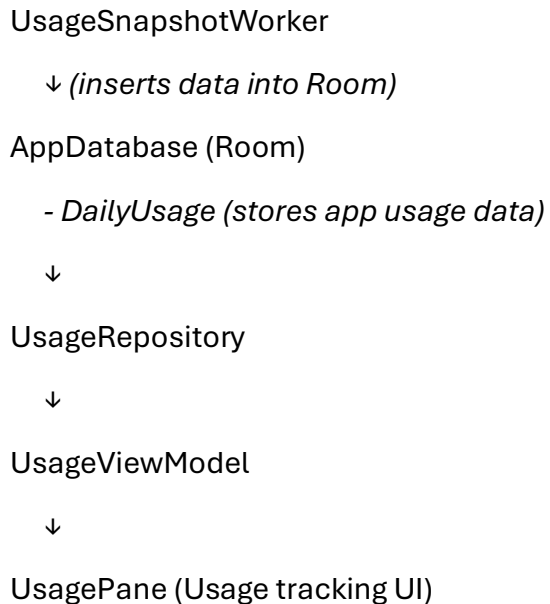
It helps visualise how often and how long apps are used, with what permissions they have access to. The user can choose to see a summary of the past 24 hours, 7 days, 30 days or 90 days.

Core Files

File	Purpose
UsagePane.kt	Jetpack Compose UI displaying usage summaries and detailed app usage per day.
UsageViewModel.kt	Handles data flow, collects usage stats from Room DB and exposes StateFlow to the UI.
UsageSnapshotWorker.kt	Background worker that periodically records app usage stats into the database.
DailyUsage.kt	Room entities storing daily usage totals and per-app sessions.
DailyUsageDao.kt	Provides DB access methods for querying usage data.

UsageRepository.kt	Abstracts database queries, used by the ViewModel.
--------------------	--

Data Flow



Process Flow:

Worker collection:

UsageSnapshotWorker runs periodically (via WorkManager) and queries Android's UsageStatsManager API.

It records total daily usage into DailyUsage and detailed session data into DailySession.

Database storage:

The data is stored in the Room DB (AppDatabase). Data older than 90 days is pruned automatically.

ViewModel exposure:

UsageViewModel retrieves the recent records via UsageRepository and exposes them as a Flow.

UI rendering:

UsagePane.kt observes this Flow using collectAsState().

It displays usage summaries, session durations, and top apps in a LazyColumn.

Notes

-If adding new usage metrics, extend UsageSnapshotWorker and add new fields to DailyUsage.

-Always keep Room schema and DAO queries in sync.

-Usage data retention (90-day cleanup) is handled by the worker automatically.

-Data is collected once per day or when manually refreshed by the user. The automatic data collection frequency can be increased or decreased and is controlled by the WorkManager scheduling in MainActivity.kt

Recent Changes Section

Purpose

The Recent Changes section on the Home Pane shows the latest permission events (grant or revoke) across installed apps.

It lets the user quickly see which apps have recently changed their permission states.

Core Files

File	Purpose
HomePane.kt	UI for the Home screen, includes the 'Recent Changes' tile.
HomeViewModel.kt	Fetches permission change data from the database and exposes it to the UI.
Permissions.kt	Room entity storing permission change logs (package name, permission, time, action).
PermissionDao.kt	Database access interface for retrieving recent changes.
PermissionRepository.kt	Collects and manages permission-related data, including tracking updates.
PermissionSnapshotWorker.kt	Periodically scans for new permission changes and logs them.

Data Flow:

PermissionSnapshotWorker

↓ *(scans apps and permissions)*

PermissionRepository

↓ *(compares new vs old)*

AppDatabase (Room)

- *PermissionSnapshot (stores current states)*

- *PermissionChange (stores grant/revoke events)*

↓

HomeViewModel



HomePane (Recent Changes UI)

Process flow:

The system periodically checks app permissions and compares them to the last recorded state. The PermissionSnapshot table captures the current permission state of each installed app and compares them to the last recorded states. If there are any changes (granted or revoked) then this is recorded in the PermissionChange table.

Any change saved into the PermissionChange table is done via PermissionRepository.

HomeViewModel retrieves the last 7 days of events (sorted by date and time).

HomePane displays them in the 'Recent Changes' card.

Each entry shows the app icon, name, permission, and time of change.

Notes:

- The recent changes section currently shows permission changes from the last 7 days. This can be changed from within PermissionRepository.kt.

- When the list is empty, the UI shows a placeholder ('No recent changes').

- Data is kept for 90 days before it is removed from both tables. This can be changed from within PermissionRepository.kt.

Risk Filter & Risk Summary Section

Purpose

The Risk Filter and Risk Summary provide visibility into detected risk events and allow users to filter, sort, and review risk levels across apps and time periods. Information displayed includes total risk count, severity distribution (e.g., Low/Medium/High/Critical), recent high-risk events, and filterable attributes (app, permission, time range, severity, status).

They help users quickly identify high-severity risks, understand risk trends over time, and narrow results to the most relevant events.

Core Files

File	Purpose
PermissionWarningUtils	Utility functions for deriving risk levels and warnings from permissions, policies, and app metadata.
HomeViewModel	Coordinates data flow, applies filters, aggregates risk stats, and exposes StateFlow for the home UI.
RiskLevelFilter	Model and helpers defining filter criteria (severity, app, permission, status, time range) and composable predicates.
AppsPane	Jetpack Compose UI that displays filtered apps, risk badges, severity breakdowns, and recent high-risk events.

Data Flow

PermissionWarningUtils / RiskLevelFilter

↓ (rules + filter criteria applied)

HomeViewModel

↓ (aggregates + exposes state)

AppsPane (Risk UI)

Process Flow:

Risk evaluation:

PermissionWarningUtils applies risk rules (e.g., permission sensitivity, policy alignment, anomalous patterns) and produces risk levels and warnings. RiskLevelFilter defines the filter criteria and predicates used by the ViewModel.

ViewModel exposure:

HomeViewModel ingests events/metadata, applies current filters, computes severity distributions and top-risk lists, and exposes them as Flow/StateFlow for the UI.

UI rendering:

AppsPane observes state via collectAsState() and presents totals, severity breakdowns, and high-risk items in a LazyColumn with filter controls.

Notes

If adding new risk rules/metrics, extend PermissionWarningUtils and update RiskLevelFilter to include the new criteria.

Keep schema/models and ViewModel computations in sync when changing filters or aggregates.

Ensure filter criteria are reflected consistently across DAO/Repository (if present), HomeViewModel, and AppsPane to align list and summary.

Support manual refresh triggers in the UI to recompute aggregates after rule or data changes.

App Filter by Permission & App Type:

Purpose:

The Permission and App Type filter lets user narrow the app list by the permission that an app requests/uses and by whether the app is a User or System app. This allows the user to quickly focus on apps with specific permissions and reduce noise by scoping to apps installed by the user only.

Core Files:

File	Purpose
AppsPane.kt	The AppsPane UI displays the applications as is where the filters are located. AppsPane hosts the filtering logic through two reactive states, permissionFiltered (permission based filtering), and finalList, which combines the filters to output a result that satsafies all selected filter options. The PermissionChipsGrid composable toggles the chip states.
AppsPaneViewModel.kt	This file stores the active states of each permission chip Boolean, these Boolean values are observed in AppsPane.kt to determine which permission filters are applied.
AppInfo.kt	AppInfo data model is used by the filters, this provides insight for the applications and which permission they are using, such as if an app is using camera, cameraPermissions Boolean will be set to true, this is done for all permissions for all apps.
MainActivityViewModel.kt	This supplies the base list of apps using appInfo. SDK flags are used to verify android version to determine specific permissions they are only available after certain android versions. For example the physical activity permission chip is only available after Android 10, if the version is below that, that filter will be ignored.

Data Flow:

Permission chip toggles (AppsPaneViewModel)

↓ Observed as StateFlow(Boolean)

AppsPane.kt then builds permissionFiltered -> then finalList

↓

Apps are rendered by AppList() with the filtered list of AppInfo

Process Flow:

- **Permission Filtering**

1. The user opens permissions from the filters, the bottom sheet lists the permissions, PermissionChipsGrid()
2. Each chip toggles a boolean inside AppsPaneViewModel
3. PermissionFiltered in AppsPane.kt reacts to these states, if no chips are selected it will return the full AppInfo list otherwise, will provide the filtered list against the AppInfo fields.

BodySensors -> bodySensorPermissions

Calendar -> calendarPermissions

- e.t.c. ""

4. SDK Gates to determine Android Version

- SdkGreaterThanTiramisu (Android 13), Photos & Video permission was changed significantly this is why this version is required.
- SdkGreaterThanQ (Android 10), Physical Activity chip, similarly this wasn't introduced in android 10, just significantly changed how the permission was handled by apps.

- **App Type Filtering**

1. User opens app type filter then selects between All, User, or System. The default setting is on All; this is a private enum for AppType in AppsPane.kt
2. Each app is included in finalList and is displayed using the filter after checking with applicationInfo.flags, this filters by:

User Application:

- `ApplicationInfo.FLAG_System == 0`

System Application:

- `ApplicationInfo.FLAG_SYSTEM != 0`

Notes:

- The default state for all permission chips is false; App Type also defaults to All, therefore all applications are displayed initially.
- Both filters use `deprivedStateOf` for efficiency in changing filters as the chip stage changes.
- SDK version, the photos & video and the physical activity are used when the correct SDK flags in `MainActivityViewModel` are true.

Permission Granted Box (HomePane.kt)

Purpose

Provides a concise overview of how many total permissions have been granted across installed apps.

This feature helps users quickly understand the current permission exposure level at a glance.

Implementation Details

- Added as a dedicated composable within HomePane.kt, linked to MainActivityViewModel.kt for live data.
- Uses `collectAsState()` to observe permission data updates.
- Includes a subtle animation using `animateColorAsState` for state transitions (e.g., changing color based on permission count).
- Integrated with risk summary logic to visually align with the *Sensitive Permission Box* and *Risk Strip*.

Data Flow

PermissionRepository.kt

↓ (fetches total granted permissions via system API)

MainActivityViewModel.kt

↓ (aggregates & exposes count as `StateFlow<Int>`)

HomePane.kt

↓ (`collectAsState` observes changes)

PermissionGrantedBox.kt

↓ (renders permission count & color animation)

User Interface

Process Flow

1. PermissionRepository queries installed apps and retrieves their granted permission counts using the system PackageManager.
2. MainActivityViewModel aggregates these values and emits them through a StateFlow.
3. HomePane.kt observes this flow and passes the value into PermissionGrantedBox.
4. The composable displays the total count and animates the box color based on thresholds (e.g., low = green, medium = orange, high = red).
5. The data updates reactively when any permission change event is detected by the worker in the background.

Sensitive Permission Box (HomePane.kt)

Purpose

Displays a summary of all high-risk or sensitive permissions currently granted, such as **Camera**, **Location**, **Contacts**, **Microphone**, or **SMS**.

The intent was to give users immediate awareness of sensitive data access on their device.

Implementation Details

- Implemented dynamic filtering within the `ViewModel` to identify apps using sensitive permissions.
- Added a red-accented UI card to visually differentiate high-risk permissions.
- The list updates in real time whenever new permissions are granted or revoked.
- Connected with `PermissionWarningUtils.kt` and `RiskLevelFilter` to maintain consistency with the global risk classification model.
- Includes a user-friendly summary text (e.g., “5 apps have access to sensitive permissions”).

Data Flow

`PermissionRepository.kt`

↓ (scans apps + permissions)

`PermissionWarningUtils.kt`

↓ (applies sensitivity rules)

`MainActivityViewModel.kt`

↓ (filters high-risk permissions & emits as `StateFlow<List<AppInfo>>`)

`HomePane.kt`

↓ (`collectAsState` observes changes)

`SensitivePermissionBox.kt`

↓ (renders sensitive app list & count)

User Interface

Process Flow

1. The `PermissionRepository` retrieves the latest permission set from all installed apps.
2. `PermissionWarningUtils.kt` classifies each permission as *normal* or *sensitive*.

3. The filtered list of apps with sensitive permissions is passed into MainActivityViewModel.
4. The ViewModel exposes this filtered list to the UI via StateFlow.
5. SensitivePermissionBox observes these updates and visually displays the sensitive app count with a red-accented card.
6. Whenever a new sensitive permission is granted or revoked, the box updates in real time.

App Permission Description (SettingsPane.kt)

Purpose

Provides clear, human-readable explanations of what each Android permission does, helping users make informed privacy decisions.

Implementation Details

- Added a structured permission-description section to SettingsPane.kt.
- Permission data is stored in a local JSON or constants file (e.g., PermissionDescriptions.kt).
- Implemented a scrollable list composable (LazyColumn) with permission name, category icon, and descriptive text.
- Designed typography hierarchy (heading → description → divider) for clarity and accessibility.
- Fully integrated with Jetpack DataStore for theme and display consistency.

Data Flow

PermissionDescriptions.kt (local static data source)



SettingsViewModel.kt (loads and caches permission info)



SettingsPane.kt



PermissionDescriptionList.kt (LazyColumn composable)



User Interface

Process Flow

1. SettingsViewModel.kt loads permission descriptions from the static file PermissionDescriptions.kt or embedded JSON.
2. The ViewModel stores the data in memory and exposes it to the UI through a state holder.
3. SettingsPane.kt observes this data and displays it using LazyColumn in PermissionDescriptionList.kt.
4. Each permission entry displays a title, category icon, and human-readable description (e.g., *Camera – allows the app to take photos or record videos*).
5. The composable respects the active app theme and updates automatically when user preferences (e.g., dark mode or high contrast) change via DataStore.

Accessibility Settings

Purpose:

These settings allow the user to personalize the app for comfort and accessibility. The features are:

- **Appearance:** Light Mode, Dark Mode & System (Matches the system theme)
- **High Contrast:** increased contrast between items with true black and white surfaces and stronger foregrounds to improve separation and increase readability
- **Text to Speech:** Shortcut to Androids accessibility settings for text to speech options as well as instructions included in the app locally on how to use the feature
- **About Us:** Pop up box which includes information about the company and developers and includes an embedded link to the company site.

Core Files:

File	Purpose
SettingsPane.kt	SettingsPane.kt is the main user interface for the setting page, this renders the settings list and items.
Theme.kt	Defines the ThemeMode: System, Light and Dark, applies dynamic color on Android12+, observes highContrastFlow and then overrides the Material3 colorScheme for high contrast in each light and dark mode.
SettingsStore.kt	SettingsStore is a datastore, in this case used for settings, themeFlow, setTheme, hihContrastFlow, and setHighContrast.
Build.gradle.kts	Declared dependencies that were used to implement these features.

Data Flow:

- **Light/Dark Mode**

SettingsPane.kt -> AppearanceSection (theme buttons)

↓ Collects SettingsStore.themeFlow

SettingsStore.setTheme writes theme_mode

↓ observed by PermiCheckTheme(), reads themeFlow

PermicheckTheme resolves dark/light

↓ Android 12+ versions use dynamic colour, otherwise regular light/dark scheme

MaterialTheme.colorScheme updates

↓

UI Recomposes based on theme selected

- **High Contrast Mode**

SettingsPane - High Contrast Switch

↓ (collects SettingsStore.highContrastFlow as StateFlow boolean

SettingsStore.setHighContrast(context, enabled) writes high_contrast

↓ this is then observed by PermiCheckTheme

PermiCheckTheme copies base scheme colours into a high contrast variant

↓ true black and white services are applied on the surface colour values

MaterialTheme.colorScheme is updated

↓

UI Recomposes based on contrast option selected

- **Text to Speech**

SettingsPane – Text to speech label

↓ (on select)

Intent(Settings.ACTION_ACCESSIBILITY_SETTINGS)

↓

Android Text to Speech Settings is opened

- **About Us**

SettingsPane – About us Label

↓ (on click)

AlertDialog (Pop up on screen)

↓ (play store link for feedback)

Sends user to the playstore to get input/feedback

Process Flow:

Theme Mode:

1. User taps desired theme mode
2. Handler calls `SettingsStore.setTheme` to persist selected mode
3. `SettingsStore.themeFlow` emits the value
4. `PermiCheckTheme` reads `themeFlow` and recomposes application
5. `MaterialTheme.colorScheme` updates icons and all screens immediately

High Contrast:

1. User toggles high contrast switch
2. Handler calls `SettingsStore.setHighContrast`
3. `SettingsStore.highContrastFlow` emits the boolean value
4. `PermiCheckTheme` applies the high contrast overrides
5. App recomposes with `highcontrast` applied

Text to Speech:

1. User taps Text to Speech item
2. Launches `Settings.ACTION_ACCESSIBILITY_SETTINGS`
3. Opens Android's native TTS Settings

About Us:

1. User taps About Us item
2. AlertDialog opens with information containing; who we are, what does the app do?, FAQs, and support
3. Embedded link takes user to playstore to provide feedback
4. Close button closes the pop up dialog box.